



Solemne 3

INDICACIONES

- Esta Solemne se realiza de forma individual.
- La solemne debe ser rendida en la plataforma <http://codetrack.cl/>
- Para entrar debe escribir su correo institucional y como password su rut (solo números).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Limpieza y resumen de un catálogo estelar con pandas.

Enunciado

Un catálogo pequeño de estrellas fue armado desde distintas fuentes y guardado en un archivo CSV. Antes de comparar las estrellas, se debe leer la tabla, inspeccionarla, eliminar mediciones incompletas y crear una columna física derivada.

El archivo CSV se encuentra en la siguiente ruta:

```
1 url = (  
2     "https://raw.githubusercontent.com/jperaltac/pcfi161/"  
3     "main/Solemnes/2026/S3/catalogo_estrellas_s3.csv"  
4 )
```

Realice las siguientes operaciones:

- Importe `numpy`, `pandas` y `matplotlib.pyplot`. Lea el archivo CSV usando `pd.read_csv(url)` y guárdelo en un DataFrame llamado `estrellas`.
- Inspeccione la tabla mostrando las primeras filas, dimensiones, tipos de datos y cantidad de valores faltantes por columna.
- Cree un DataFrame llamado `estrellas_limpias` eliminando solo las filas que tengan datos faltantes en `temperatura_K`, `radio_Rsol`, `distancia_pc` o `magnitud_V`. Use `copy()` al final.
- En `estrellas_limpias`, cree la columna `luminosidad_rel` usando

$$L_{\text{rel}} = R^2 \left(\frac{T}{5778} \right)^4,$$

donde R es `radio_Rsol` y T es `temperatura_K`.

- Agrupe `estrellas_limpias` por tipo y guarde ese grupo en una variable.
- Antes de calcular estadísticas, cree una copia de `estrellas_limpias` sin la columna `nombre`. Agrupe esa nueva tabla por tipo y calcule cuántas estrellas hay en cada tipo, el promedio por tipo y la desviación estándar por tipo. Imprima estos resultados.
- Haga un gráfico de dispersión simple usando dos columnas del DataFrame. Grafique `temperatura_K` en el eje horizontal y `luminosidad_rel` en el eje vertical. Agregue etiquetas, título y grilla.

Solución.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 url = (
6     "https://raw.githubusercontent.com/jperaltac/pcfi161/"
7     "main/Solemnes/2026/S3/catalogo_estrellas_s3.csv"
8 )
9
10 estrellas = pd.read_csv(url)
11
12 print(estrellas.head())
13 print("Dimensiones:", estrellas.shape)
14 print(estrellas.dtypes)
15 print("Valores faltantes:")
16 print(estrellas.isna().sum())
17
18 columnas_necesarias = [
19     "temperatura_K",
20     "radio_Rsol",
21     "distancia_pc",
22     "magnitud_V"
23 ]
24
25 estrellas_limpias = estrellas.dropna(
26     subset=columnas_necesarias
27 ).copy()
28
29 estrellas_limpias["luminosidad_rel"] = (
30     estrellas_limpias["radio_Rsol"]**2 *
31     (estrellas_limpias["temperatura_K"] / 5778)**4
32 )
33
34 grupo_tipo = estrellas_limpias.groupby("tipo")
35
36 estrellas_sin_nombre = estrellas_limpias.drop(columns=["nombre"]).copy()
37 grupo_tipo_sin_nombre = estrellas_sin_nombre.groupby("tipo")
38
39 cantidad_por_tipo = grupo_tipo_sin_nombre["tipo"].count()
40 promedio_por_tipo = grupo_tipo_sin_nombre.mean()
41 desviacion_por_tipo = grupo_tipo_sin_nombre.std()
42
43 print(cantidad_por_tipo)
44 print(promedio_por_tipo)
45 print(desviacion_por_tipo)
46
47 plt.figure(figsize=(7, 5))
48 plt.scatter(
49     estrellas_limpias["temperatura_K"],
50     estrellas_limpias["luminosidad_rel"]
51 )
52
53 plt.xlabel("Temperatura [K]")
54 plt.ylabel("Luminosidad relativa")
55 plt.title("Temperatura versus luminosidad relativa")
56 plt.grid(True, alpha=0.3)
57 plt.show()

```

2. Ordenamiento y búsqueda de períodos orbitales.

Enunciado

Un equipo observa exoplanetas y registra sus períodos orbitales en días:

```
periodos = [12.4, 3.1, 8.7, 25.0, 1.9, 14.2, 6.5, 18.3].
```

Utilice los algoritmos entregados a continuación. No debe escribir otro algoritmo de ordenamiento ni otro algoritmo de búsqueda. En una parte del ejercicio se le pedirá modificar `bubble_sort` para contar comparaciones e intercambios:

```
1 def bubble_sort(lista):
2     n = len(lista)
3     for i in range(n - 1):
4         for j in range(n - 1 - i):
5             if lista[j] > lista[j + 1]:
6                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
7     return lista
8
9 def binary_search(lista_ordenada, objetivo):
10     low, high = 0, len(lista_ordenada) - 1
11
12     while low <= high:
13         mid = (low + high) // 2
14
15         if lista_ordenada[mid] == objetivo:
16             return mid
17         elif objetivo < lista_ordenada[mid]:
18             high = mid - 1
19         else:
20             low = mid + 1
21
22     return -1
```

Realice las siguientes operaciones:

- Defina la lista `periodos`.
- Modifique la función `bubble_sort` entregada para que también cuente cuántas comparaciones entre elementos realiza y cuántos intercambios hace. La función modificada debe devolver tres valores: la lista ordenada, el número de comparaciones y el número de intercambios. No modifique `binary_search`.
- Ordene los períodos de menor a mayor usando la función `bubble_sort` modificada. Debe ordenar una copia de la lista original, no la lista original directamente.
- Imprima la lista original, la lista ordenada, el número de comparaciones y el número de intercambios.
- Determine el período mínimo y el período máximo usando la lista ordenada.
- Use `binary_search` para buscar el período 14.2 en la lista ordenada. Imprima su posición si fue encontrado.
- Use `binary_search` para buscar el período 10.0. Si no aparece, imprima un mensaje claro indicando que no fue encontrado.

Solución.

```
1 periodos = [12.4, 3.1, 8.7, 25.0, 1.9, 14.2, 6.5, 18.3]
2
3 def bubble_sort(lista):
4     n = len(lista)
5     comparaciones = 0
6     intercambios = 0
7
8     for i in range(n - 1):
9         for j in range(n - 1 - i):
10             comparaciones = comparaciones + 1
11             if lista[j] > lista[j + 1]:
```

```

12         lista[j], lista[j + 1] = lista[j + 1], lista[j]
13         intercambios = intercambios + 1
14
15     return lista, comparaciones, intercambios
16
17 def binary_search(lista_ordenada, objetivo):
18     low, high = 0, len(lista_ordenada) - 1
19
20     while low <= high:
21         mid = (low + high) // 2
22
23         if lista_ordenada[mid] == objetivo:
24             return mid
25         elif objetivo < lista_ordenada[mid]:
26             high = mid - 1
27         else:
28             low = mid + 1
29
30     return -1
31
32 periodos_ordenados, comparaciones, intercambios = bubble_sort(
33     periodos.copy()
34 )
35
36 print("Lista original:", periodos)
37 print("Lista ordenada:", periodos_ordenados)
38 print("Comparaciones:", comparaciones)
39 print("Intercambios:", intercambios)
40
41 periodo_minimo = periodos_ordenados[0]
42 periodo_maximo = periodos_ordenados[-1]
43
44 print("Periodo mínimo:", periodo_minimo)
45 print("Periodo máximo:", periodo_maximo)
46
47 posicion_142 = binary_search(periodos_ordenados, 14.2)
48 if posicion_142 == -1:
49     print("El período 14.2 no fue encontrado")
50 else:
51     print("Posición de 14.2:", posicion_142)
52
53 posicion_100 = binary_search(periodos_ordenados, 10.0)
54 if posicion_100 == -1:
55     print("El período 10.0 no fue encontrado")
56 else:
57     print("Posición de 10.0:", posicion_100)

```

3. Ajuste cuadrático y estimación de un umbral.

Enunciado

En una calibración de laboratorio se mide la respuesta R de un detector para distintas dosis d . Los datos medidos son:

$$d = [0, 1, 2, 3, 4, 5, 6],$$

$$R = [1,2, 2,0, 4,1, 6,9, 10,8, 16,2, 22,1].$$

Se desea construir un modelo cuadrático

$$R(d) = ad^2 + bd + c$$

y usarlo para estimar cuándo la respuesta alcanza un umbral de seguridad

$$R = 12.$$

Realice las siguientes operaciones:

- (a) Importe numpy y matplotlib.pyplot. Defina los arreglos d y R usando np.array.
- (b) Use np.polyfit(d, R, deg=2) para obtener los coeficientes a, b y c.
- (c) Calcule los valores ajustados sobre los mismos datos medidos y guárdelos en R_fit.
- (d) Calcule los residuos y el coeficiente R^2 :

$$\text{residuos} = R - R_{\text{fit}},$$

$$SS_{\text{res}} = \sum_i (R_i - R_{\text{fit},i})^2, \quad SS_{\text{tot}} = \sum_i (R_i - \bar{R})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

- (e) Use np.roots para resolver cuando el modelo ajustado alcanza $R = 12$. Para ello debe resolver

$$ad^2 + bd + c - 12 = 0.$$

- (f) Filtre las raíces: conserve solo soluciones reales dentro del rango medido, es decir, entre 0 y 6.
- (g) Grafique los datos medidos con plt.scatter, el ajuste cuadrático con plt.plot usando una grilla fina para la dosis, y la línea horizontal $R = 12$. Para esta grilla fina, cree un arreglo con np.linspace para la variable independiente. Agregue etiquetas, leyenda y active la grilla del gráfico.
- (h) Imprima a, b, c, R^2 y la dosis estimada para alcanzar el umbral. Agregue una frase que explique por qué no basta con tomar cualquier raíz entregada por np.roots.

Solución.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 d = np.array([0, 1, 2, 3, 4, 5, 6])
5 R = np.array([1.2, 2.0, 4.1, 6.9, 10.8, 16.2, 22.1])
6
7 a, b, c = np.polyfit(d, R, deg=2)
8
9 R_fit = a*d**2 + b*d + c
10
11 residuos = R - R_fit
12 SS_res = np.sum(residuos**2)
13 SS_tot = np.sum((R - np.mean(R))**2)
14 R2 = 1 - SS_res/SS_tot
15
16 coef_umbral = [a, b, c - 12]
17 raices = np.roots(coef_umbral)
18
19 soluciones = []
20 for r in raices:
21     if abs(r.imag) < 1e-10 and 0 <= r.real <= 6:
22         soluciones.append(r.real)
23
24 d_fino = np.linspace(0, 6, 200)
25 R_fino = a*d_fino**2 + b*d_fino + c
26
27 plt.figure(figsize=(7, 5))
28 plt.scatter(d, R, label="datos medidos")
29 plt.plot(d_fino, R_fino, "r-", label="ajuste cuadrático")
30 plt.axhline(12, color="gray", linestyle="--", label="umbral R=12")
31 plt.xlabel("Dosis d")
32 plt.ylabel("Respuesta R")
33 plt.title("Ajuste cuadrático y umbral de seguridad")
34 plt.legend()
35 plt.grid(True, alpha=0.3)
36 plt.show()

```

```
37
38 print(f"a = {a:.4f}")
39 print(f"b = {b:.4f}")
40 print(f"c = {c:.4f}")
41 print(f"R2 = {R2:.4f}")
42
43 if len(soluciones) > 0:
44     print(f"Dosis estimada para R=12: {soluciones[0]:.3f}")
45 else:
46     print("No hay una solución real dentro del rango medido")
47
48 print("No basta con tomar cualquier raíz de np.roots:")
49 print("hay que descartar raíces complejas y raíces fuera del rango físico.")
```